MatchFIT MVP Backend

Requirements Specification — One-Day Activation MVP

A lean backend supporting **one complete operational journey**: register → create player profile → add to today's session → admin check-in → coach assessment → award points → division, rating and leaderboard update. **No** computer vision, RFID, payments, subscriptions or league automation for Day 1.

1. Tech Stack

Layer	Choice
Framework	Next.js (App Router, server routes under src/app/api)
Language	TypeScript
Database	Supabase PostgreSQL
Auth	Supabase Auth
File storage	Supabase Storage (bucket: player-photos)
Validation	Zod (on every mutation route)
QR generation	qrcode.react (client-side, encodes a URL only)
Deployment	Vercel

Install:

```
npm install next react react-dom @supabase/supabase-js @supabase/ssr zod qrcode.react
npm install -D typescript @types/node @types/react @types/react-dom eslint eslint-config-next
```

Environment variables (.env.local):

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY= # server-only, NEVER in browser code
NEXT_PUBLIC_APP_URL=http://localhost:3000
ADMIN_EMAIL=
```

2. Scope

In scope: participant registration, player profiles, admin login, session management, attendance / check-in, initial assessment, manual point awards, 10 divisions, leaderboard, session groups / teams, five-pillar tracking, profile photo storage, basic audit trail.

Out of scope: computer vision, automated match analysis, RFID, payments, subscriptions, auto promotion/relegation, AI recommendations, video storage, multi-location, social feed, messaging, injury diagnosis.

3. Database (Supabase — 001_matchfit_mvp.sql)

Table	Purpose
divisions	10 seeded divisions; new players default to Division 10
players	Profile + consent + emergency contact + injury notes
admins	Linked to auth.users; roles: admin / coach / staff

Table	Purpose
sessions	Date, times, capacity, status, current pillar
session_registrations	Unique (session, player)
attendance	Status, checked_in_at, checked_in_by
assessments	6 coach scores (1–10) + computed overall_rating
point_transactions	Action, points, computed total_points (store as transactions)
session_groups	Group / team / station
pillar_completion	One row per pillar (1–5)

Plus: **updated_at** triggers, a **player_leaderboard** view, and **RLS enabled on all tables**.

4. Player Rating Formula

Six coach scores (1–10), weighted. **overall_rating = round(weightedScore × 10)** — computed server-side only; never accept it from the client.

Attribute	Weight
Movement	15%
Stamina	15%
Ball control	20%
Passing	15%
1v1	15%
Game awareness	20%

5. Points Configuration (server-side)

Action	Points
attendance	100
five_pillars_complete	50
goal	20
assist	15
successful_one_v_one	10
tackle_or_interception	10
match_win	25
fair_play	15
coach_bonus	5 (allow 5–25)

Points are derived server-side. Reject arbitrary client point values (except the coach-bonus range).

6. Required API Routes

All responses use the envelope **{ success, data }** or **{ success, error: { code, message } }**.

Method & Route	Access	Purpose
POST /api/players	Public	Create participant + register for today's open session
GET /api/players/:id	Public link	Profile, division, assessment, points, attendance, pillars
GET /api/sessions/today	Public	Today's open/active session + schedule + counts
PATCH /api/sessions/:id/attendance	Admin	Check-in; award attendance points once
POST /api/sessions/:id/assessments	Admin/Coach	Record 6 scores; backend computes rating
POST /api/sessions/:id/points	Admin/Coach	Insert validated point transaction
PATCH /api/sessions/:id/groups	Admin	Assign group / team / station
PATCH /api/sessions/:id/pillars	Admin/Coach	Mark pillar; award 5-pillar bonus once
GET /api/leaderboard	Public	?period=all|today and ?division=N

7. Security & Rules

- **Prevent duplicates:** phone number, session registration, attendance rows, repeated attendance points, repeated 5-pillar bonus, duplicate division seeds — use DB unique constraints **and** app checks.
- Public users may only register, view their own profile (secure link), and view today's session and the leaderboard.
- Only authenticated staff may check-in, assess, award points, change groups/divisions, mark pillars, or close sessions.
- **Never expose** injury notes, emergency contacts, phone or email in public / leaderboard responses.
- Service-role key: server routes only. QR codes encode a URL only (/check-in/{player_id}) — never personal data. Consent must be true at registration.

8. Team Ownership

Developer	Area of Ownership
Dev 1 — Backend Lead	Supabase setup, schema/migrations, shared types, auth, deployment, PR review
Dev 2 — Player Service	Registration, profile fetch, photo upload, duplicate-phone handling
Dev 3 — Session Service	Today's session, registration, attendance, QR check-in
Dev 4 — Assessment Service	Assessment validation, rating formula, coach notes
Dev 5 — Points & Divisions	Point transactions, division seed, leaderboard, duplicate-award protection
Dev 6 — Admin Integration	Admin API consumption, groups, pillars, end-to-end testing, error/loading states

9. Definition of Done

- Production DB live; frontend connected to real data.
- A participant can register in under 2 minutes; check-in works from a phone.
- Assessments, points and leaderboard update without refresh errors.
- No sensitive fields publicly exposed; service-role key not in browser.
- Production build succeeds; full journey tested with 10+ sample players.

10. Local Setup

```
npm install
cp .env.local.example .env.local # then fill in Supabase keys
npm run dev # http://localhost:3000

# before merging:
npm run lint && npm run typecheck && npm run build
```